UNIVERSIDADE FEDERAL DO CEARÁ

Test Smells em projetos OpenSource

Disciplina: Qualidade de Software
Equipe: João Vitor Rodrigues Laurindo e Maria Letícia dos Santos Dantas
Projeto: NVIDIA NeMo Guardrails
Fork: https://github.com/letdnts/Guardrails

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Registro 01 — Sleepy Test Responsável: João Vitor Rodrigues Laurindo. Data: 26/10/2026. Arquivo: tests/test_cache_lfu.py Classe/método: TestLFUCacheStatsLogging.test_stats_logging_with_empty_cache Ocorrência original: time.sleep(0.2), linha 451 da medição inicial.
Minhas principais dificuldades na remoção do code smell são: Ponto técnico tratado: apenas apagar o sleep faria a verificação ocorrer antes do intervalo de logging. Foi necessário controlar o relógio usado pelo cache, preservando as cinco asserções originais.
Usei o CATSPY para: Ex: Refatoração, Definição, ETC. Consulta à definição, aos efeitos e às orientações de refatoração do Sleepy Test. O catálogo recomenda evitar esperas explícitas e apresenta o uso de mocks como alternativa. Neste caso, foi simulado o relógio para preservar a verificação do intervalo de logging. Página consultada: https://cat-spy.vercel.app/smells/Sleepy%20Test
Eu estou usando os seguintes métodos de refatoração para remover o code smell: Substituição de time.sleep(0.2) por unittest.mock.patch no objeto time do módulo nemoguardrails.llm.cache.lfu. O relógio passa de 1000,0 para 1000,2 segundos sem espera real, ultrapassando o intervalo de logging de 0,1 segundo. As cinco asserções originais foram mantidas e foi adicionada a verificação de que o primeiro acesso não gera log. Validação: 44 testes aprovados em tests/test_cache_lfu.py; TEMPY confirmou a ausência de Sleepy Test no método e a redução de 7 para 6 ocorrências desse tipo no arquivo. Ruff (lint e formatação) aprovado. Commit local: e6523547.

UNIVERSIDADE FEDERAL DO CEARÁ

Eu estou atualmente trabalhando na refatoração do seguinte code smell:
Registro 02 — Sleepy Test (2 ocorrências)
Responsável: João Vitor Rodrigues Laurindo. Data: 26/10/2026.
Arquivo: tests/test_cache_lfu.py
Classe/método: TestLFUCacheStatsLogging.test_periodic_stats_logging
Ocorrências originais: duas chamadas time.sleep(0.6), linhas 426 e 437 da medição inicial.

Minhas principais dificuldades na remoção do code smell são:
Ponto técnico tratado: controlar o relógio desde as primeiras inserções no cache, preservando as quatro verificações de contagem de logs (0, 1, 1 e 2). A operação sem avanço de tempo deve continuar sem produzir um novo log. Refatoração implementada e validada com apoio do Codex.

Usei o CATSPY para: Ex: Refatoração, Definição, ETC.
A página Sleepy Test foi consultada para confirmar os efeitos das esperas explícitas e orientar sua substituição por mecanismos controlados, como mocks. A aplicação neste teste foi simular o relógio mantendo as verificações do intervalo de logging.
Página consultada: https://cat-spy.vercel.app/smells/Sleepy%20Test

Eu estou usando os seguintes métodos de refatoração para remover o code smell:
Substituição das duas chamadas time.sleep(0.6) por um relógio simulado com unittest.mock.patch no objeto time do módulo nemoguardrails.llm.cache.lfu. O teste começa em 1000,0 s e avança 0,6 s em dois momentos, mantendo o intervalo de logging de 0,5 s e as quatro asserções originais.
Validação deste commit: 44 testes aprovados no arquivo; TEMPY confirmou a remoção das duas ocorrências no método (Sleepy Test no arquivo: 6 para 4). Ruff aprovado.
Commit local: e824f5f6.

Eu estou atualmente trabalhando na refatoração do seguinte code smell:
Registro 03 — Sleepy Test (1 ocorrência)
Responsável: João Vitor Rodrigues Laurindo. Data: 26/10/2026.
Arquivo: tests/test_cache_lfu.py
Classe/método: TestLFUCacheStatsLogging.test_stats_logging_with_full_cache
Ocorrência original: time.sleep(0.2), linha 480 da medição inicial.

UNIVERSIDADE FEDERAL DO CEARÁ

Minhas principais dificuldades na remoção do code smell são:
Ponto técnico tratado: manter o cenário de cache cheio, com quatro inserções em uma capacidade de três itens e uma remoção por falta de espaço. O relógio precisa ser controlado antes das inserções para garantir que só haja logging depois do avanço de 0,2 s. As três asserções originais foram preservadas.

Usei o CATSPY para: Ex: Refatoração, Definição, ETC.
Consulta à definição, aos efeitos e à orientação de substituir esperas explícitas por mecanismos controlados. Foi aplicado um mock de relógio para manter a verificação do intervalo de logging sem uma espera real.
Página consultada: https://cat-spy.vercel.app/smells/Sleepy%20Test

Eu estou usando os seguintes métodos de refatoração para remover o code smell:
Substituição de time.sleep(0.2) por um relógio simulado com unittest.mock.patch no objeto time do módulo nemoguardrails.llm.cache.lfu. O teste configura 1000,0 s, preenche o cache e causa uma remoção; depois avança para 1000,2 s. Foram mantidas as verificações de tamanho, remoções e inserções e adicionadas verificações de ausência de log antes do intervalo e de uma única chamada após ele.
Validação: 44 testes aprovados no arquivo; TEMPY confirmou a remoção da ocorrência no método (Sleepy Test no arquivo: 4 para 3). Ruff aprovado.
Commit local: c255e774. Com o registro 01 (1 ocorrência) e o registro 02 (2 ocorrências), foram concluídas as 4 ocorrências de Sleepy Test da parte de João.

Eu estou atualmente trabalhando na refatoração do seguinte code smell:
Registro 04 — Verbose Test (1 ocorrência). João Vitor Rodrigues Laurindo — 26/10/2026. Arquivo: tests/test_cache_lfu.py; TestLFUCache.test_complex_eviction_scenario (linhas 131–180 antes da alteração).

Minhas principais dificuldades na remoção do code smell são:
Ponto de atenção técnico: manter as frequências a=3, b=c=2 e d=1 e a ordem das inserções, preservando as verificações originais das duas remoções. Refatoração e validação realizadas com auxílio do Codex.

Usei o CATSPY para: Ex: Refatoração, Definição, ETC.
Consultar a definição, os efeitos na legibilidade e a recomendação de dividir métodos longos em métodos menores e focados. Página consultada: https://cat-spy.vercel.app/smells/Verbose%20Test

Eu estou usando os seguintes métodos de refatoração para remover o code smell:
Divisão do cenário em test_complex_eviction_removes_lowest_frequency e test_complex_eviction_preserves_higher_frequencies, com a preparação comum em

_cache_with_multiple_frequency_levels. As asserções originais foram preservadas; foi acrescentada a verificação de tamanho após a primeira remoção. Validação: 45 testes do arquivo aprovados; TEMPY confirmou a ausência de Verbose Test nos métodos resultantes; Ruff aprovado. Commit: 991f934c.

Eu estou atualmente trabalhando na refatoração do seguinte code smell:
Registro 05 — Verbose Test (1 ocorrência). João Vitor Rodrigues Laurindo — 26/10/2026.
Arquivo: tests/test_cache_lfu.py;
TestLFUCacheThreadSafety.test_concurrent_reads_writes (linhas 746–793 antes das alterações).

Minhas principais dificuldades na remoção do code smell são:
Ponto de atenção técnico: Preservar as dez threads, as cem operações por thread e a distinção entre chaves privadas e compartilhadas. O cache mantém capacidade 2000 para evitar remoções durante este cenário. Refatoração e validação realizadas com auxílio do Codex.

Usei o CATSPY para:
Consultar a definição, os efeitos na legibilidade e a recomendação de dividir métodos longos em métodos menores e focados. Página consultada:
https://cat-spy.vercel.app/smells/Verbose%20Test

Eu estou usando os seguintes métodos de refatoração para remover o code smell:
Extração de _read_write_worker, separando a carga concorrente das verificações de integridade, funcionamento posterior do cache e estatísticas. As quatro asserções originais e a propagação das exceções por future.result() foram mantidas. Validação: 45 testes do arquivo aprovados; TEMPY confirmou a ausência de Verbose Test nos métodos resultantes; Ruff aprovado. Commit: 9cf2e4ac.

Eu estou atualmente trabalhando na refatoração do seguinte code smell:
Registro 06 — Verbose Test (1 ocorrência). João Vitor Rodrigues Laurindo — 26/10/2026.
Arquivo: tests/test_cache_lfu.py;
TestLFUCacheThreadSafety.test_get_or_compute_thread_safety (linhas 892–941 antes das alterações).

Minhas principais dificuldades na remoção do code smell são:
Ponto de atenção técnico: Manter dez solicitações simultâneas para a mesma chave e comprovar que o cálculo ocorre somente uma vez, sem perder a verificação do valor devolvido a todos os participantes. Refatoração e validação realizadas com auxílio do Codex.

UNIVERSIDADE FEDERAL DO CEARÁ

Usei o CATSPY para:
Consultar a definição, os efeitos na legibilidade e a recomendação de dividir métodos longos em métodos menores e focados. Página consultada: https://cat-spy.vercel.app/smells/Verbose%20Test

Eu estou usando os seguintes métodos de refatoração para remover o code smell:
Extração de _compute_shared_key_concurrently para preparar e executar a carga assíncrona. O teste concentra as três asserções originais: resultados iguais, uma única computação e valor computed_value_1. Validação: 45 testes do arquivo aprovados; TEMPY confirmou a ausência de Verbose Test nos métodos resultantes; Ruff aprovado. Commit: 81263cd8.

Eu estou atualmente trabalhando na refatoração do seguinte code smell:
Registro 07 — Verbose Test (1 ocorrência). João Vitor Rodrigues Laurindo — 26/10/2026. Arquivo: tests/test_cache_lfu.py; TestLFUCacheThreadSafety.test_concurrent_operations_with_evictions (linhas 1130–1178 antes das alterações).

Minhas principais dificuldades na remoção do code smell são:
Ponto de atenção técnico: Distinguir uma remoção legítima de corrupção de dados. Foi preservada a tolerância a valores ausentes após remoções concorrentes, assim como as dez threads, as cem operações e a capacidade 50. Refatoração e validação realizadas com auxílio do Codex.

Usei o CATSPY para:
Consultar a definição, os efeitos na legibilidade e a recomendação de dividir métodos longos em métodos menores e focados. Página consultada: https://cat-spy.vercel.app/smells/Verbose%20Test

Eu estou usando os seguintes métodos de refatoração para remover o code smell:
Extração de _evicting_worker para concentrar as inserções e acessos concorrentes. O teste mantém as quatro asserções sobre integridade, tamanho e estatísticas de remoções e inserções, além de aguardar todos os futures. Validação: 45 testes do arquivo aprovados; TEMPY confirmou a ausência de Verbose Test nos métodos resultantes; Ruff aprovado. Commit: 96013d21.

Eu estou atualmente trabalhando na refatoração do seguinte code smell:
Registro 08 — Verbose Test (1 ocorrência). João Vitor Rodrigues Laurindo — 26/10/2026. Arquivo: benchmark/tests/test_mock_response_data.py; TestGenerateChunkLatencies.test_generate_latencies_multiple_chunks_no_seed (linhas 375–422 antes das alterações).

UNIVERSIDADE FEDERAL DO CEARÁ

Minhas principais dificuldades na remoção do code smell são:
Ponto de atenção técnico: Manter a ordem TTFT/ITL e todos os argumentos verificados dos mocks ao separar a distribuição normal da aplicação de limites. Cada teste recebe uma fixture independente, evitando dependência entre os cenários.

Usei o CATSPY para:
Consultar a definição, os efeitos na legibilidade e a recomendação de dividir métodos longos em métodos menores e focados. Página consultada: https://cat-spy.vercel.app/smells/Verbose%20Test

Eu estou usando os seguintes métodos de refatoração para remover o code smell:
Extração da fixture multiple_chunk_generation_no_seed e divisão em dois testes focados: parâmetros das distribuições normais e limites de clipping. Foram preservadas todas as verificações de argumentos, quantidades de chamadas, ausência de seed e cinco latências. Validação: 77 testes dos dois arquivos aprovados; TEMPY confirmou a ausência de Verbose Test nos métodos resultantes; Ruff aprovado. Commit: 2b59c67d.

Eu estou atualmente trabalhando na refatoração do seguinte code smell:
Registro 09 — Redundant Print (1 ocorrência). João Vitor Rodrigues Laurindo — 26/10/2026. Arquivo: tests/cli/test_migration.py; TestColang1SyntaxConversion.test_complex_conversion; print originalmente na linha 257.

Minhas principais dificuldades na remoção do code smell são:
Ponto de atenção técnico: remover somente a saída de depuração e manter a asserção que compara a conversão com o resultado esperado. O cenário já estava marcado com skip porque a conversão ainda não foi implementada; essa condição foi mantida.

Usei o CATSPY para:
Consultar a definição e a orientação de remover impressões desnecessárias, usando asserções para avaliar automaticamente o resultado. Página consultada: https://cat-spy.vercel.app/smells/Redundant%20Print

Eu estou usando os seguintes métodos de refatoração para remover o code smell:
Remoção de print(convert_colang_1_syntax(input_lines)), preservando assert convert_colang_1_syntax(input_lines) == expected_output. Assim, o resultado continua sendo verificado pela asserção, sem uma chamada adicional usada apenas para impressão. Validação: 53 testes aprovados e 1 ignorado, como antes da alteração; TEMPY confirmou Redundant Print de 1 para 0 no arquivo; Ruff aprovado. Commit: df14b83d.

UNIVERSIDADE FEDERAL DO CEARÁ

Eu estou atualmente trabalhando na refatoração do seguinte code smell:
Registro 10 — Conditional Test Logic (2 ocorrências). João Vitor Rodrigues Laurindo — 26/10/2026. Arquivo: tests/test_cache_lfu.py; TestLFUCacheThreadSafety.test_thread_safe_size_operations; if nas linhas 960 e 964 antes destas refatorações.

Minhas principais dificuldades na remoção do code smell são:
Ponto de atenção técnico: Manter as verificações de tamanho mínimo, capacidade máxima e correspondência entre tamanho zero e is_empty. As falhas dentro das threads precisam chegar ao teste principal; future.result() continua fazendo essa propagação.

Usei o CATSPY para:
Consultar a definição e os efeitos das condições dentro dos testes e a recomendação de tornar explícitos os comportamentos esperados. Neste caso, os ramos que acumulavam erros foram substituídos por asserções diretas. Página consultada: https://cat-spy.vercel.app/smells/Conditional%20Test%20Logic

Eu estou usando os seguintes métodos de refatoração para remover o code smell:
Substituição dos dois if que preenchiam uma lista de erros por assertGreaterEqual(size, 0), assertLessEqual(size, 100) e assertEqual(size == 0, is_empty). Foram mantidas as dez threads e as cem operações por thread. A lista de erros e a asserção agregada foram substituídas pelas verificações diretas, com mensagens que identificam a thread. Validação: 45 testes do arquivo aprovados; Ruff aprovado; TEMPY confirmou a redução de 46 para 44 ocorrências de Conditional Test Logic no arquivo. Commit: 89214dee.

Eu estou atualmente trabalhando na refatoração do seguinte code smell:
Registro 11 — Conditional Test Logic (3 ocorrências). João Vitor Rodrigues Laurindo — 26/10/2026. Arquivo: tests/test_cache_lfu.py; TestLFUCacheThreadSafety.test_concurrent_contains_operations; if nas linhas 991, 1002 e 1008 antes destas refatorações.

Minhas principais dificuldades na remoção do code smell são:
Ponto de atenção técnico: Preservar a verificação de chaves existentes e inexistentes e justificar a exigência de que uma chave recém-inserida continue presente. O cenário usa no máximo 550 chaves e um cache de capacidade 1000, portanto não deve ocorrer remoção por falta de espaço.

Usei o CATSPY para:
Consultar a definição e os efeitos das condições dentro dos testes e a recomendação de tornar explícitos os comportamentos esperados. Neste caso, os ramos que acumulavam erros foram substituídos por asserções diretas. Página consultada: https://cat-spy.vercel.app/smells/Conditional%20Test%20Logic

UNIVERSIDADE FEDERAL DO CEARÁ

Eu estou usando os seguintes métodos de refatoração para remover o code smell:
Substituição dos três if por duas chamadas assertTrue para chaves existentes e recém-inseridas e uma chamada assertFalse para chaves inexistentes. As listas de erros e avisos foram removidas; future.result() propaga as falhas das cinco threads. A ausência de uma chave nova, antes apenas armazenada como aviso não utilizado, agora reprova o teste. Validação: 45 testes do arquivo aprovados; Ruff aprovado; TEMPY confirmou a redução de 44 para 41 ocorrências de Conditional Test Logic no arquivo. Somadas ao registro 10, são cinco ocorrências corrigidas. Commit: 8ee7a6af.

Eu estou atualmente trabalhando na refatoração do seguinte code smell:
Registro 12 — Conditional Test Logic (1 ocorrência). João Vitor Rodrigues Laurindo — 26/10/2026. Arquivo: tests/test_cache_lfu.py; TestLFUCache.test_repeated_puts_same_key; condição if self.cache.track_stats, originalmente na linha 285.

Minhas principais dificuldades na remoção do code smell são:
Ponto de atenção técnico: a configuração padrão do setUp desativa as estatísticas, por isso a asserção condicional sobre as dez atualizações não era executada. Era necessário ativar explicitamente as estatísticas neste cenário, preservando as verificações de tamanho e do último valor armazenado.

Usei o CATSPY para:
Aplicar a orientação de explicitar o cenário e o resultado esperado, evitando que uma condição determine se uma asserção será executada. Referência consultada: https://cat-spy.vercel.app/smells/Conditional%20Test%20Logic

Eu estou usando os seguintes métodos de refatoração para remover o code smell:
Configuração explícita de LFUCache(3, track_stats=True) e substituição do if por assertEqual(self.cache.get_stats()["updates"], 10), executada sempre. A variável initial_stats, que não era utilizada, foi removida. As dez atualizações, as verificações de tamanho igual a 1 e a verificação do valor final value9 foram mantidas. Validação: 45 testes do arquivo aprovados; Ruff aprovado; TEMPY confirmou a redução de 41 para 40 ocorrências de Conditional Test Logic no arquivo. Commit: 6d4741d0.

Eu estou atualmente trabalhando na refatoração do seguinte code smell:
Registro 13 — Conditional Test Logic (1 ocorrência). João Vitor Rodrigues Laurindo — 26/10/2026. Arquivo: tests/test_cache_lfu.py; TestLFUCacheThreadSafety.test_concurrent_stats_operations; if originalmente na linha 843, antes destas duas refatorações.

UNIVERSIDADE FEDERAL DO CEARÁ

Minhas principais dificuldades na remoção do code smell são: Ponto de atenção técnico: Preservar as cinco threads, as cinquenta iterações e as operações de inserção, acerto e falta em cada iteração. As três verificações da estrutura das estatísticas precisam ser executadas sem depender do índice da iteração.
Usei o CATSPY para: Aplicar a orientação de tornar explícitos o cenário e o resultado esperado, removendo condições que selecionam verificações ou acumulam erros. Referência consultada: https://cat-spy.vercel.app/smells/Conditional%20Test%20Logic
Eu estou usando os seguintes métodos de refatoração para remover o code smell: Remoção do if i % 10 == 0 e execução direta de assertIsInstance(stats, dict), assertIn("hits", stats) e assertIn("misses", stats) após cada grupo de operações. As verificações passam de cinco para cinquenta vezes por thread. As asserções finais sobre acertos, faltas e inserções foram mantidas. Validação: 45 testes do arquivo aprovados; Ruff aprovado; TEMPY confirmou a redução de 40 para 39 ocorrências de Conditional Test Logic no arquivo. Commit: 088d2179.

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Registro 14 — Conditional Test Logic (1 ocorrência). João Vitor Rodrigues Laurindo — 26/10/2026. Arquivo: tests/test_cache_lfu.py; TestLFUCacheThreadSafety.test_concurrent_reset_stats; if originalmente na linha 1020, antes destas duas refatorações.
Minhas principais dificuldades na remoção do code smell são: Ponto de atenção técnico: Manter a detecção de qualquer estatística negativa e a propagação de falhas das threads pelo future.result(). Com o rastreamento ativado, get_stats() retorna um dicionário não vazio de valores numéricos, permitindo verificar diretamente o menor valor.
Usei o CATSPY para: Aplicar a orientação de tornar explícitos o cenário e o resultado esperado, removendo condições que selecionam verificações ou acumulam erros. Referência consultada: https://cat-spy.vercel.app/smells/Conditional%20Test%20Logic
Eu estou usando os seguintes métodos de refatoração para remover o code smell: Substituição do if any(...) e da lista de erros por assertGreaterEqual(min(stats.values()), 0), com mensagem que identifica a thread e as estatísticas. Se o menor valor for não negativo, todos os valores do snapshot também serão. Foram mantidas as cinco threads, as cinquenta iterações e a reinicialização das estatísticas a cada dez iterações. Validação: 45 testes do arquivo aprovados; Ruff aprovado; TEMPY confirmou a redução de 39 para 38 ocorrências de Conditional Test Logic no arquivo. Commit: a2121b78.

UNIVERSIDADE FEDERAL DO CEARÁ

Eu estou atualmente trabalhando na refatoração do seguinte code smell:
Registro 15 — Conditional Test Logic (1 ocorrência). João Vitor Rodrigues Laurindo — 26/10/2026. Arquivo: tests/test_cache_lfu.py; TestLFUCacheStatsLogging.test_stats_log_format_percentages; if hits > 0 or misses > 0, originalmente na linha 599.

Minhas principais dificuldades na remoção do code smell são:
Ponto de atenção técnico: o cenário sem acessos estava na tabela de casos, mas a condição impedia sua asserção. Como log_stats_now() gera o log sempre que o rastreamento está ativado, esse caso também deve verificar a taxa esperada de 0.00%.

Usei o CATSPY para:
Aplicar a orientação de explicitar os resultados esperados sem uma condição que decida se a asserção será executada. Referência consultada: https://cat-spy.vercel.app/smells/Conditional%20Test%20Logic

Eu estou usando os seguintes métodos de refatoração para remover o code smell:
Remoção do if e execução direta da verificação do percentual nos seis cenários existentes. Foi adicionada mock_log.assert_called_once() antes de ler a mensagem, garantindo uma chamada de log por cenário. Os casos continuam cobrindo zero acessos, somente acertos, somente faltas, 50.00%, 66.67% e 99.00%. Validação: 45 testes do arquivo aprovados; Ruff aprovado; TEMPY confirmou a redução de 38 para 37 ocorrências de Conditional Test Logic no arquivo. Commit: a90a5570.

Eu estou atualmente trabalhando na refatoração do seguinte code smell:
Registro 16 — Conditional Test Logic (1 ocorrência). João Vitor Rodrigues Laurindo — 26/10/2026. Arquivo: tests/test_cache_lfu.py; TestLFUCacheStatsLogging.test_stats_log_format_percentages; if hits > 0, originalmente na linha 587.

Minhas principais dificuldades na remoção do code smell são:
Ponto de atenção técnico: garantir que hit_key esteja disponível para todos os acessos bem-sucedidos sem condicionar a preparação à quantidade de acertos. A inserção inicial deve ocorrer antes da reinicialização dos contadores de cada cenário, para preservar as quantidades de acertos e faltas usadas no cálculo dos percentuais.

Usei o CATSPY para:
Aplicar a orientação de tornar a preparação e os resultados esperados explícitos, removendo a escolha condicional do caminho de execução do teste. Referência consultada: https://cat-spy.vercel.app/smells/Conditional%20Test%20Logic

UNIVERSIDADE FEDERAL DO CEARÁ

Eu estou usando os seguintes métodos de refatoração para remover o code smell: Preparação de hit_key uma única vez antes do laço dos seis cenários e remoção do if hits > 0. Cada cenário continua reinicializando as estatísticas antes de gerar os acessos. O laço de acertos executa naturalmente zero vezes quando hits é zero. Foram preservados os seis casos de percentual e as verificações de chamada e conteúdo do log. Validação: 45 testes do arquivo aprovados; Ruff aprovado; TEMPY confirmou a redução de 37 para 36 ocorrências de Conditional Test Logic no arquivo. Commit: 7dfbb388. Com os registros anteriores, foram concluídas as 20 ocorrências planejadas para João: 10 Conditional Test Logic, 5 Verbose Test, 4 Sleepy Test e 1 Redundant Print.

Eu estou atualmente trabalhando na refatoração do seguinte code smell:

Registro 17 — Redundant Print (1 ocorrência).

Responsável: Maria Letícia. Data: 25/10/2026.

Arquivo: tests/test_cache_lfu.py

Classe/método: TestLFUCacheThreadSafety.test_concurrent_contains_operations

Ocorrência original: chamada print de depuração, linha 1039 da medição inicial.

Minhas principais dificuldades na remoção do code smell são:

Ponto técnico tratado: o print era apenas uma saída de diagnóstico e não alimentava nenhuma verificação, mas estava em um teste com várias threads. A dificuldade foi removê-lo sem alterar a parte concorrente nem as verificações de contains. Esse mesmo método também aparece no Registro 11, que trata de outro smell (Conditional Test Logic); por isso a mudança ficou restrita à linha da impressão.

Usei o CATSPY para:

Consultei a definição do Redundant Print e a orientação de remover impressões desnecessárias: como o teste faz parte de uma suíte automatizada, o resultado deve ser avaliado por asserções, e não pela leitura da saída no terminal.

Página consultada: https://cat-spy.vercel.app/smells/Redundant%20Print

Eu estou usando os seguintes métodos de refatoração para remover o code smell:

Remoção da chamada print de diagnóstico de test_concurrent_contains_operations. Nenhuma asserção foi alterada ou removida; o comportamento do teste continua definido apenas pelas suas verificações.

Validação: O teste foi aprovado nas medições antes e depois; o TEMPY deixou de apontar o Redundant Print deste método (linha 1039).

Commit: refactor(tests): remove redundant print from test_concurrent_contains_operations (330097f).

UNIVERSIDADE FEDERAL DO CEARÁ

Eu estou atualmente trabalhando na refatoração do seguinte code smell:

Registro 18 — Conditional Test Logic (1 ocorrência).

Responsável: Maria Letícia. Data: 26/10/2026.

Arquivo: tests/test_embeddings_providers_mock.py

Classe/método: TestCohereEmbeddingModelMocked.test_all_predefined_models

Ocorrência original: laço for sobre o dicionário models_to_test, linha 184 da medição inicial.

Minhas principais dificuldades na remoção do code smell são:

Ponto técnico tratado: o laço verificava cinco modelos da Cohere (embed-v4.0, embed-english-v3.0, embed-english-light-v3.0, embed-multilingual-v3.0 e embed-multilingual-light-v3.0) em um único teste, de modo que a falha em um deles escondia o resultado dos seguintes. Era preciso manter os cinco pares modelo/dimensão, as duas asserções de cada iteração (embedding_size e model) e o mock de cohere montado com patch.dict em sys.modules.

Usei o CATSPY para:

Consultei a definição do Conditional Test Logic, que inclui laços como o for entre as estruturas condicionais que não devem ficar no corpo do teste, e a recomendação de tornar explícito o comportamento esperado. A decisão foi transformar cada iteração em um caso de teste próprio.

Página consultada: https://cat-spy.vercel.app/smells/Conditional%20Test%20Logic

Eu estou usando os seguintes métodos de refatoração para remover o code smell:

Substituição do for por @pytest.mark.parametrize(“model_name, expected_size”, COHERE_MODELS), com a lista de cinco tuplas COHERE_MODELS declarada acima da classe. A criação do mock e o patch.dict continuam dentro do teste, e as asserções de embedding_size e model foram mantidas, agora executadas como cinco casos independentes.

Validação: O teste, que era 1 caso, passou a rodar 5 casos (um por modelo), todos aprovados; o TEMPY deixou de apontar o Conditional Test Logic da linha 184.

Commit: refactor(tests): parametrize cohere predefined models test (102596e).

Eu estou atualmente trabalhando na refatoração do seguinte code smell:

Registro 19 — Conditional Test Logic (1 ocorrência).

Responsável: Maria Letícia. Data: 26/10/2026.

Arquivo: tests/test_embeddings_providers_mock.py

Classe/método: TestOpenAIEmbeddingModelMocked.test_all_predefined_models

Ocorrência original: laço for sobre o dicionário models_to_test, linha 330 da medição inicial.

UNIVERSIDADE FEDERAL DO CEARÁ

Minhas principais dificuldades na remoção do code smell são:

Ponto técnico tratado: o laço percorria três modelos da OpenAI (text-embedding-ada-002 e text-embedding-3-small, com 1536 dimensões, e text-embedding-3-large, com 3072). A preparação do mock define mock_openai.__version__ = “1.0.0”, e esse detalhe precisava continuar presente em cada caso gerado pela parametrização.

Usei o CATSPY para:

Usei a definição do Conditional Test Logic para confirmar que o for dentro do teste é uma lógica condicional a ser retirada, e a orientação de expressar cada cenário esperado de forma direta, sem iteração.

Página consultada: https://cat-spy.vercel.app/smells/Conditional%20Test%20Logic

Eu estou usando os seguintes métodos de refatoração para remover o code smell:

Substituição do for por @pytest.mark.parametrize(“model_name, expected_size”, OPENAI_MODELS), com a lista de três tuplas declarada acima da classe. O mock com __version__, o patch.dict e as asserções de embedding_size e model permaneceram no teste, agora executados como três casos separados.

Validação: O teste, que era 1 caso, passou a rodar 3 casos (um por modelo), todos aprovados; o TEMPY deixou de apontar o Conditional Test Logic da linha 330.

Commit: refactor(tests): parametrize openai predefined models test (6dfe48f).

Eu estou atualmente trabalhando na refatoração do seguinte code smell:

Registro 20 — Conditional Test Logic (1 ocorrência).

Responsável: Maria Letícia. Data: 26/10/2026.

Arquivo: tests/test_embeddings_providers_mock.py

Classe/método: TestCohereEmbeddingModelMocked.test_import_error_when_cohere_not_installed

Ocorrência original: if que verifica o módulo em sys.modules antes do del, linha 79 da medição inicial.

Minhas principais dificuldades na remoção do code smell são:

Ponto técnico tratado: o if existia para apagar o módulo do provedor de sys.modules e forçar a reimportação, garantindo que o ImportError fosse disparado mesmo que o módulo já estivesse em cache. Apenas trocar por um del direto causaria KeyError quando o módulo ainda não tivesse sido importado, e a limpeza precisava continuar isolada para não afetar outros testes.

Usei o CATSPY para:

Consultei a definição do Conditional Test Logic para confirmar que um if de preparação também conta como lógica condicional no teste. A decisão foi trocar o ramo por uma operação que já trata os dois casos (módulo presente ou ausente).

Página consultada: https://cat-spy.vercel.app/smells/Conditional%20Test%20Logic

UNIVERSIDADE FEDERAL DO CEARÁ

Eu estou usando os seguintes métodos de refatoração para remover o code smell:

Substituição de if ... del sys.modules[...] por sys.modules.pop("nemoguardrails.embeddings.providers.cohere", None), colocado dentro do patch.dict e antes do pytest.raises. O valor padrão None evita erro quando o módulo não está carregado, e o patch.dict restaura sys.modules ao sair. O ImportError esperado ("Could not import cohere") continua sendo verificado.

Validação: O teste foi aprovado antes e depois; o TEMPY deixou de apontar o Conditional Test Logic da linha 79. O TEMPY continua apontando Unknown Test neste teste, mas esse alerta já existia antes da refatoração e não foi tratado neste commit.

Commit: refactor(tests): use sys.modules.pop in cohere import-error test (c2c4065).

Eu estou atualmente trabalhando na refatoração do seguinte code smell:

Registro 21 — Conditional Test Logic (1 ocorrência).

Responsável: Maria Letícia. Data: 26/10/2026.

Arquivo: tests/test_embeddings_providers_mock.py

Classe/método: TestOpenAIEmbeddingModelMocked.test_import_error_when_openai_not_installed

Ocorrência original: if que verifica o módulo em sys.modules antes do del, linha 231 da medição inicial.

Minhas principais dificuldades na remoção do code smell são:

Ponto técnico tratado: o mesmo padrão do registro anterior, agora para o provedor da OpenAI, com uma verificação adicional no final: o ImportError capturado deve ter como causa (__cause__) outro ImportError. Era preciso manter essa asserção fora do bloco pytest.raises, como no original.

Usei o CATSPY para:

Reaproveitei a leitura do Conditional Test Logic feita no registro anterior: o if de preparação deve sair do teste e ser substituído por uma operação sem ramificação.

Página consultada: https://cat-spy.vercel.app/smells/Conditional%20Test%20Logic

Eu estou usando os seguintes métodos de refatoração para remover o code smell:

Substituição do if/del por sys.modules.pop("nemoguardrails.embeddings.providers.openai", None), dentro do patch.dict e antes do pytest.raises. O match "Could not import openai" e a asserção sobre exc_info.value.__cause__ foram mantidos.

Validação: O teste foi aprovado antes e depois; o TEMPY deixou de apontar o Conditional Test Logic da linha 231.

Commit: refactor(tests): use sys.modules.pop in openai import-error test (0e9b833).

UNIVERSIDADE FEDERAL DO CEARÁ

Eu estou atualmente trabalhando na refatoração do seguinte code smell:

Registro 22 — Conditional Test Logic (1 ocorrência).

Responsável: Maria Letícia. Data: 26/10/2026.

Arquivo: tests/test_embeddings_providers_mock.py

Classe/método: TestAzureEmbeddingModelMocked.test_import_error_when_openai_not_installed

Ocorrência original: if que verifica o módulo em sys.modules antes do del, linha 379 da medição inicial.

Minhas principais dificuldades na remoção do code smell são:

Ponto técnico tratado: o método tem o mesmo nome do teste da classe da OpenAI, e a ausência simulada é do pacote openai, mas o módulo que precisa ser reimportado é o do provedor Azure. Era necessário apontar o pop para nemoguardrails.embeddings.providers.azureopenai, e não para o módulo da OpenAI.

Usei o CATSPY para:

Usei a definição do Conditional Test Logic para tratar este caso do mesmo modo que os anteriores, trocando a condição por uma operação única.

Página consultada: https://cat-spy.vercel.app/smells/Conditional%20Test%20Logic

Eu estou usando os seguintes métodos de refatoração para remover o code smell:

Substituição do if/del por sys.modules.pop(“nemoguardrails.embeddings.providers.azureopenai”, None), dentro do patch.dict que define openai como None. O pytest.raises com “Could not import openai” continua verificando o erro esperado.

Validação: O teste foi aprovado antes e depois; o TEMPY deixou de apontar o Conditional Test Logic da linha 379. O TEMPY continua apontando Unknown Test neste teste, mas esse alerta já existia antes da refatoração e não foi tratado neste commit.

Commit: refactor(tests): use sys.modules.pop in azure import-error test (1707914).

Eu estou atualmente trabalhando na refatoração do seguinte code smell:

Registro 23 — Conditional Test Logic (1 ocorrência).

Responsável: Maria Letícia. Data: 26/10/2026.

Arquivo: tests/test_embeddings_providers_mock.py

Classe/método: TestGoogleEmbeddingModelMocked.test_import_error_when_google_genai_not_installed

Ocorrência original: if que verifica o módulo em sys.modules antes do del, linha 604 da medição inicial.

Minhas principais dificuldades na remoção do code smell são:

UNIVERSIDADE FEDERAL DO CEARÁ

Ponto técnico tratado: neste teste o patch.dict simula a ausência de dois módulos ao mesmo tempo (google e google.genai definidos como None). A remoção do módulo do provedor precisava ocorrer depois desse patch e antes do pytest.raises, para a reimportação encontrar os módulos ausentes.

Usei o CATSPY para:

Consultei novamente a definição do Conditional Test Logic e mantive a mesma decisão dos registros 20 a 22, para que os quatro testes de ImportError fiquem com a mesma estrutura.

Página consultada: https://cat-spy.vercel.app/smells/Conditional%20Test%20Logic

Eu estou usando os seguintes métodos de refatoração para remover o code smell:

Substituição do if/del por sys.modules.pop("nemoguardrails.embeddings.providers.google", None), dentro do patch.dict e antes do pytest.raises. O match "Could not import google-genai" foi mantido.

Validação: O teste foi aprovado antes e depois; o TEMPY deixou de apontar o Conditional Test Logic da linha 604. O TEMPY continua apontando Unknown Test neste teste, mas esse alerta já existia antes da refatoração e não foi tratado neste commit.

Commit: refactor(tests): use sys.modules.pop in google import-error test (44fbf3a).

Eu estou atualmente trabalhando na refatoração do seguinte code smell:

Registro 24 — Conditional Test Logic (1 ocorrência).

Responsável: Maria Letícia. Data: 27/10/2026.

Arquivo: tests/test_embeddings_providers_mock.py

Classe/método: TestAzureEmbeddingModelMocked.test_all_predefined_models

Ocorrência original: laço for sobre o dicionário models_to_test, linha 472 da medição inicial.

Minhas principais dificuldades na remoção do code smell são:

Ponto técnico tratado: o dicionário tinha um único modelo (text-embedding-ada-002, com 1536 dimensões), então o ganho da parametrização é pequeno; ainda assim o for precisava sair do teste. A asserção deste provedor usa o atributo embedding_model, e não model como nos demais, e essa diferença foi preservada.

Usei o CATSPY para:

Usei a definição do Conditional Test Logic para justificar a remoção do for mesmo com um único elemento, já que a lista parametrizada facilita incluir novos modelos sem alterar o corpo do teste.

Página consultada: https://cat-spy.vercel.app/smells/Conditional%20Test%20Logic

Eu estou usando os seguintes métodos de refatoração para remover o code smell:

Substituição do for por @pytest.mark.parametrize("model_name, expected_size", AZURE_MODELS), com a lista de uma tupla declarada acima da classe. As asserções de embedding_size e embedding_model foram mantidas.

Validação: O caso parametrizado (text-embedding-ada-002, 1536) foi aprovado; o TEMPY deixou de apontar o Conditional Test Logic da linha 472.

Commit: refactor(tests): parametrize azure predefined models test (c8b6ce1).

Eu estou atualmente trabalhando na refatoração do seguinte code smell:

Registro 25 — Conditional Test Logic (1 ocorrência).

Responsável: Maria Letícia. Data: 27/10/2026.

Arquivo: tests/test_embeddings_providers_mock.py

Classe/método: TestGoogleEmbeddingModelMocked.test_all_predefined_models

Ocorrência original: laço for sobre o dicionário models_to_test, linha 740 da medição inicial.

Minhas principais dificuldades na remoção do code smell são:

Ponto técnico tratado: o teste cobre um único modelo (gemini-embedding-001, com 3072 dimensões) e monta um mock em dois níveis: mock_genai_module.genai aponta para mock_genai, e o patch.dict registra tanto google quanto google.genai. Essa estrutura precisava permanecer idêntica em cada caso parametrizado.

Usei o CATSPY para:

Apliquei a mesma orientação do Conditional Test Logic dos registros 18, 19 e 24, retirando o laço do corpo do teste.

Página consultada: https://cat-spy.vercel.app/smells/Conditional%20Test%20Logic

Eu estou usando os seguintes métodos de refatoração para remover o code smell:

Substituição do for por @pytest.mark.parametrize("model_name, expected_size", GOOGLE_MODELS), com a lista de uma tupla declarada acima da classe. O mock em dois níveis, o patch.dict e as asserções de embedding_size e model foram mantidos.

Validação: O caso parametrizado (gemini-embedding-001, 3072) foi aprovado; o TEMPY deixou de apontar o Conditional Test Logic da linha 740.

Commit: refactor(tests): parametrize google predefined models test (3fa9c6a).

Eu estou atualmente trabalhando na refatoração do seguinte code smell:

Registro 26 — Conditional Test Logic (2 ocorrências).

Responsável: Maria Letícia. Data: 27/10/2026.

Arquivo: tests/test_generation_options.py

Classe/método: test_generation_response_with_rail_outcome_is_json_serializable

UNIVERSIDADE FEDERAL DO CEARÁ

Ocorrências originais: cadeias if/elif/else na preparação e na extração do resultado, linhas 292 e 301 da medição inicial.

Minhas principais dificuldades na remoção do code smell são:

Ponto técnico tratado: o parâmetro surface (activated_rails, internal_events, selected_output_vars e all_output_vars) controlava duas cadeias condicionais independentes: uma para gerar a resposta, com log ou com output_vars, e outra para extrair o valor serializado. Era preciso manter o parametrize por outcome (_RAIL_OUTCOME_CASES), o helper _json_round_trip e a comparação com _expected_serialized_outcome em todos os cenários.

Usei o CATSPY para:

Consultei a definição do Conditional Test Logic e a recomendação de tornar explícito o comportamento esperado. Como cada valor de surface seguia um caminho próprio, a decisão foi dar a cada caminho o seu teste.

Página consultada: https://cat-spy.vercel.app/smells/Conditional%20Test%20Logic

Eu estou usando os seguintes métodos de refatoração para remover o code smell:

Divisão do teste em quatro: test_generation_response_activated_rails_with_rail_outcome_is_json_serializable, test_generation_response_internal_events_with_rail_outcome_is_json_serializable, test_generation_response_selected_output_vars_with_rail_outcome_is_json_serializable e test_generation_response_all_output_vars_with_rail_outcome_is_json_serializable. Todos mantêm o parametrize por outcome e a mesma comparação final, sem if/elif.

Validação: Os 12 casos (4 cenários × 3 outcomes: allow, block e transform) foram aprovados, o mesmo total do teste original; o TEMPY deixou de apontar as duas ocorrências (linhas 292 e 301).

Commit: refactor(tests): split generation response serialization test by surface (5063181).

Eu estou atualmente trabalhando na refatoração do seguinte code smell:

Registro 27 — Conditional Test Logic (1 ocorrência).

Responsável: Maria Letícia. Data: 27/10/2026.

Arquivo: tests/test_generation_options.py

Classe/método: test_generation_log_with_rail_outcome_is_json_serializable

Ocorrência original: if/else sobre log_field na extração do resultado, linha 272 da medição inicial.

Minhas principais dificuldades na remoção do code smell são:

Ponto técnico tratado: o parâmetro log_field (activated_rails ou internal_events) decidia, por meio de um if/else, qual caminho do log seria lido: o retorno da primeira ação executada ou o evento da ação localizado por _get_outcome_action_event. Os dois caminhos e a comparação com _expected_serialized_outcome precisavam ser preservados.

UNIVERSIDADE FEDERAL DO CEARÁ

Usei o CATSPY para:

Usei a definição do Conditional Test Logic e a orientação de tornar o comportamento esperado explícito para decidir separar os dois caminhos em testes distintos.

Página consultada: https://cat-spy.vercel.app/smells/Conditional%20Test%20Logic

Eu estou usando os seguintes métodos de refatoração para remover o code smell:

Divisão em test_generation_log_activated_rails_with_rail_outcome_is_json_serializable e test_generation_log_internal_events_with_rail_outcome_is_json_serializable, cada um com o parametrize por outcome e o seu caminho de asserção, sem condicional.

Validação: Os 6 casos (2 cenários × 3 outcomes) foram aprovados, o mesmo total do teste original; o TEMPY deixou de apontar o Conditional Test Logic da linha 272.

Commit: refactor(tests): split generation log serialization test by log field (926b286).

Eu estou atualmente trabalhando na refatoração do seguinte code smell:

Registro 28 — Sleepy Test (3 ocorrências).

Responsável: Maria Letícia. Data: 27/10/2026.

Arquivo: tests/test_cache_lfu.py

Classe/método: TestLFUCacheThreadSafety.test_concurrent_clear_operations

Ocorrências originais: três chamadas time.sleep (writer, clearer e reader), linhas 811, 816 e 823 da medição inicial.

Minhas principais dificuldades na remoção do code smell são:

Ponto técnico tratado: os três sleeps tinham papéis diferentes. Os do writer e do reader espalhavam as operações no tempo, e o do clearer espaçava as cinco limpezas. Apenas apagá-los faria as threads terminarem rápido demais e as limpezas poderiam não ocorrer durante as escritas. A alternativa prevista no roteiro que segui usava um semáforo liberado a cada 20 escritas, mas exigia um if no writer e criaria um novo alerta de Conditional Test Logic; por isso fiz de outra forma.

Usei o CATSPY para:

Consultei a definição do Sleepy Test e a orientação de evitar esperas explícitas, substituindo-as por um mecanismo de sincronização controlado. Também usei o catálogo para não trocar um smell por outro: a versão com if foi descartada por introduzir Conditional Test Logic.

Página consultada: https://cat-spy.vercel.app/smells/Sleepy%20Test

Eu estou usando os seguintes métodos de refatoração para remover o code smell:

Remoção dos três time.sleep e uso de um threading.Semaphore(0) chamado write_done. O writer faz write_done.release() após cada escrita, sem condicional, e o clearer faz assert write_done.acquire(timeout=5) antes de cada uma das cinco chamadas a clear(), de modo que a limpeza só ocorre depois de uma escrita e o teste não fica preso se algo falhar. Em contrapartida, as cinco limpezas acontecem logo após as primeiras escritas, e

não espaçadas ao longo das cem. A verificação final de que o cache continua funcional (put e get de final_key) foi mantida, assim como o import time, usado por outros testes do arquivo.

Validação: O teste foi aprovado antes e depois; o TEMPY deixou de apontar os três Sleepy Test (linhas 811, 816 e 823). O total de alertas de Conditional Test Logic neste método permaneceu em 5 (antes e depois), portanto nenhum smell novo foi introduzido.

Commit: refactor(tests): replace sleeps with semaphore in test_concurrent_clear_operations (1d01749).

Eu estou atualmente trabalhando na refatoração do seguinte code smell:

Registro 29 — Sleepy Test (1 ocorrência).

Responsável: Maria Letícia. Data: 27/10/2026.

Arquivo: tests/test_hf_classifier.py

Classe/método: TestPipelineConcurrency.test_concurrent_callers_wait_for_first_load

Ocorrência original: time.sleep(0.1) em slow_pipeline, linha 877 da medição inicial.

Minhas principais dificuldades na remoção do code smell são:

Ponto técnico tratado: o sleep simulava um carregamento lento do pipeline, para que o segundo chamador chegasse enquanto o primeiro ainda carregava. Com um tempo fixo, essa sobreposição era apenas esperada, e não garantida. Era preciso manter a verificação de que os dois chamadores recebem o mesmo mock_pipe.

Usei o CATSPY para:

Consultei a definição do Sleepy Test e a orientação de substituir esperas por mecanismos controlados. Aqui, o carregamento passou a terminar somente quando o teste decide liberá-lo.

Página consultada: https://cat-spy.vercel.app/smells/Sleepy%20Test

Eu estou usando os seguintes métodos de refatoração para remover o code smell:

Criação de release_load = threading.Semaphore(0) e troca de time.sleep(0.1) por assert release_load.acquire(timeout=5) dentro de slow_pipeline. Depois de iniciar a segunda thread, o teste chama release_load.release(), para que o carregamento só termine depois de a segunda thread ter sido iniciada. As asserções results[0] is mock_pipe e results[1] is mock_pipe foram mantidas, assim como o import time do arquivo.

Validação: O teste foi aprovado antes e depois; o TEMPY deixou de apontar o Sleepy Test da linha 877.

Commit: refactor(tests): replace sleep with semaphore in test_concurrent_callers_wait_for_first_load (2dc3908).

Eu estou atualmente trabalhando na refatoração do seguinte code smell:

UNIVERSIDADE FEDERAL DO CEARÁ

Registro 30 — Verbose Test (1 ocorrência).

Responsável: Maria Letícia. Data: 27/10/2026.

Arquivo: tests/test_llmrails.py

Classe/método: test_register_methods_return_self

Ocorrência original: teste com linhas 1007–1065 da medição inicial.

Minhas principais dificuldades na remoção do code smell são:

Ponto técnico tratado: um único teste verificava sete métodos register_* (register_action, register_action_param, register_filter, register_output_parser, register_prompt_context, register_embedding_search_provider e register_embedding_provider). Dois deles dependem de classes auxiliares (DummyEmbeddingProvider e DummyEmbeddingModel) e de imports locais, que precisavam acompanhar o teste correspondente. Todas as mensagens das asserções deviam ser mantidas.

Usei o CATSPY para:

Consultei a definição do Verbose Test, seus efeitos na legibilidade e a recomendação de dividir métodos longos em testes menores e focados.

Página consultada: https://cat-spy.vercel.app/smells/Verbose%20Test

Eu estou usando os seguintes métodos de refatoração para remover o code smell:

Divisão em sete testes, um por método (por exemplo, test_register_action_returns_self), com a fixture rails_for_registration, que cria o LLMRails a partir de RailsConfig.from_content(config={“models”: []}) e FakeLLMModel(responses=[]). Cada teste mantém a asserção result is rails_for_registration com a mensagem original. O test_method_chaining, que vem logo depois, não foi alterado.

Validação: O teste único deu lugar a sete testes, todos aprovados; o TEMPY deixou de apontar o Verbose Test (linhas 1007–1065).

Commit: refactor(tests): split test_register_methods_return_self into one test per method (b2b5fa7).

Eu estou atualmente trabalhando na refatoração do seguinte code smell:

Registro 31 — Verbose Test (1 ocorrência).

Responsável: Maria Letícia. Data: 27/10/2026.

Arquivo: tests/test_clavata_models.py

Classe/método: TestCreateJobResponse.test_model_validate_valid_json

Ocorrência original: teste com linhas 229–276 da medição inicial.

Minhas principais dificuldades na remoção do code smell são:

Ponto técnico tratado: cerca de metade do teste era um JSON aninhado (job, results, report e sectionEvaluationReports), e o restante era a conferência campo a campo de duas seções (DogBarking e CatMeowing). Era preciso reduzir o tamanho sem deixar de

verificar status, quantidade de resultados, resultado do relatório e nome, mensagem e resultado de cada seção.

Usei o CATSPY para:

Consultei a definição do Verbose Test e a recomendação de simplificar testes longos, extraindo o que é construção de dados para fora do corpo do teste.

Página consultada: https://cat-spy.vercel.app/smells/Verbose%20Test

Eu estou usando os seguintes métodos de refatoração para remover o code smell:

Criação da fábrica _job_response_json(sections, status, result), declarada acima de TestCreateJobResponse, que monta o JSON a partir de uma lista de tuplas (nome, mensagem, resultado). No teste, a mesma lista sections serve para montar a resposta e para conferi-la: [(s.name, s.message, s.result) for s in report.sectionEvaluationReports] == sections. Assim, nome, mensagem, resultado e quantidade de seções continuam verificados, além do status e do resultado do relatório.

Validação: O teste foi aprovado antes e depois; o TEMPY deixou de apontar o Verbose Test (linhas 229–276).

Commit: refactor(tests): build job response payload with a factory in test_model_validate_valid_json (0e6c88e).

Eu estou atualmente trabalhando na refatoração do seguinte code smell:

Registro 32 — Verbose Test (1 ocorrência).

Responsável: Maria Letícia. Data: 27/10/2026.

Arquivo: tests/test_clavata_models.py

Classe/método: TestCreateJobResponse.test_model_validate_complete_job_response

Ocorrência original: teste com linhas 301–358 da medição inicial.

Minhas principais dificuldades na remoção do code smell são:

Ponto técnico tratado: o teste tinha três seções (Label1, Label2 e Label3, com os resultados OUTCOME_TRUE, OUTCOME_FALSE e OUTCOME_FAILED), cada uma conferida em três asserções separadas. A dificuldade foi comprimir essas nove verificações sem perder a correspondência de cada campo com a sua seção.

Usei o CATSPY para:

Apliquei a mesma recomendação do Verbose Test do registro 31, desta vez reaproveitando a fábrica já criada.

Página consultada: https://cat-spy.vercel.app/smells/Verbose%20Test

Eu estou usando os seguintes métodos de refatoração para remover o code smell:

Reuso de _job_response_json com uma lista de três tuplas. A conferência campo a campo foi substituída pela comparação da lista de tuplas (nome, mensagem, resultado)

com a lista original, que cobre a quantidade e todos os campos. As asserções de status, quantidade de resultados e resultado do relatório foram mantidas.

Validação: O teste foi aprovado antes e depois; o TEMPY deixou de apontar o Verbose Test (linhas 301–358).

Commit: refactor(tests): build job response payload with a factory in test_model_validate_complete_job_response (1f60a9a).

Eu estou atualmente trabalhando na refatoração do seguinte code smell:

Registro 33 — Verbose Test (1 ocorrência).

Responsável: Maria Letícia. Data: 27/10/2026.

Arquivo: tests/evaluate/test_evaluate_cli_and_data.py

Classe/método: test_topical_command_invokes_evaluation_class

Ocorrência original: teste com linhas 52–99 da medição inicial.

Minhas principais dificuldades na remoção do code smell são:

Ponto técnico tratado: uma longa lista de argumentos da linha de comando (config, verbose, test-percentage, max-tests-intent, max-samples-intent, results-frequency, sim-threshold, random-seed e output-dir) ocupava boa parte do teste. Ao extraí-la, era importante não esconder a intenção do teste: o que a CLI deve repassar à classe de avaliação continua visível nas asserções.

Usei o CATSPY para:

Consultei a definição do Verbose Test e a recomendação de separar a construção dos dados de entrada do que o teste realmente verifica.

Página consultada: https://cat-spy.vercel.app/smells/Verbose%20Test

Eu estou usando os seguintes métodos de refatoração para remover o code smell:

Extração da lista para a função _topical_cli_args(config, output), declarada acima do teste, que passou a chamar runner.invoke(evaluate.app, _topical_cli_args(config, output)). As asserções sobre exit_code, set_verbose, os argumentos recebidos pela classe de avaliação e a chamada a evaluate_topical_rails foram mantidas sem alteração.

Validação: O teste foi aprovado antes e depois; o TEMPY deixou de apontar o Verbose Test (linhas 52–99).

Commit: refactor(tests): extract CLI args helper in test_topical_command_invokes_evaluation_class (2dabd22).

Eu estou atualmente trabalhando na refatoração do seguinte code smell:

Registro 34 — Conditional Test Logic (2 ocorrências).

Responsável: Maria Letícia. Data: 27/10/2026.

UNIVERSIDADE FEDERAL DO CEARÁ

Arquivo: tests/test_action_dispatcher.py

Classe/método: test_load_all_actions_registers_builtin_library_action_refs_lazily

Ocorrências originais: compreensão que monta action_modules, linhas 118 e 119 da medição inicial.

Minhas principais dificuldades na remoção do code smell são:

Ponto técnico tratado: as duas ocorrências estavam na compreensão que monta o conjunto action_modules, com um laço sobre os registros do catálogo, outro sobre as referências de ação e um condicional para manifestos sem actions. Ao mexer nesse trecho, era essencial preservar a verificação de carregamento tardio: o módulo nemoguardrails.library.content_safety.actions não pode estar em sys.modules depois de criar o dispatcher nem depois de ler registered_actions, e só deve aparecer quando a ação content_safety_check_input é acessada.

Usei o CATSPY para:

Consultei a definição do Conditional Test Logic e a orientação de manter o corpo do teste direto, sem estruturas de controle. A decisão foi mover a lógica de montagem do conjunto para funções auxiliares, mantendo o teste focado nas asserções.

Página consultada: https://cat-spy.vercel.app/smells/Conditional%20Test%20Logic

Eu estou usando os seguintes métodos de refatoração para remover o code smell:

Extração de _get_manifest_action_refs(record), que devolve () quando manifest.actions é None, e de _get_manifest_action_modules(catalog), que monta o conjunto de módulos. O teste passou a chamar apenas action_modules = _get_manifest_action_modules(catalog). Todas as asserções originais, inclusive as que comprovam o carregamento tardio do módulo, foram mantidas.

Validação: O teste foi aprovado antes e depois; o TEMPY deixou de apontar as duas ocorrências de Conditional Test Logic (linhas 118 e 119).

Commit: refactor(tests): extract manifest action module helper (95fd210).

Eu estou atualmente trabalhando na refatoração do seguinte code smell:

Registro 35 — Verbose Test (1 ocorrência).

Responsável: Maria Letícia. Data: 27/10/2026.

Arquivo: tests/test_patronus_evaluate_api.py

Classe/método: test_patronus_evaluate_api_success_strategy_all_pass

Ocorrência original: teste com linhas 58–120 da medição inicial.

Minhas principais dificuldades na remoção do code smell são:

Ponto técnico tratado: em cerca de 60 linhas, o teste reunia a configuração YAML do avaliador, a criação do TestChat com três respostas simuladas do modelo, o registro das ações, a resposta simulada da API com dois avaliadores e a conversa com o chat. Esses blocos se repetem nos outros testes do arquivo. A dificuldade foi extrair a parte repetida

sem esconder o cenário específico: estratégia all_pass com os dois avaliadores retornando pass: True e a resposta final do bot, “Hi there! How are you doing?”.

Usei o CATSPY para:

Consultei a definição do Verbose Test e a recomendação de reduzir a repetição e o tamanho do teste, extraindo a preparação comum para uma função auxiliar.

Página consultada: https://cat-spy.vercel.app/smells/Verbose%20Test

Eu estou usando os seguintes métodos de refatoração para remover o code smell:

Extração da configuração YAML compartilhada e da criação do TestChat para uma função auxiliar, mantendo no teste a resposta simulada da API e a interação com o chat.

Validação: O teste foi aprovado antes e depois; o TEMPY deixou de apontar o Verbose Test (linhas 58–120) e também o Non-Functional Statement da linha 60 neste teste. O Unknown Test continua sendo apontado, mas já existia antes da refatoração.

Commit: refactor(tests): reduce verbose Patronus API test (2c217fb).